ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

Студент: Александров Ярослав Дмитриевич

Группа: 25212

Тема: Высокоуровневая работа с периферийными устройствами

1. ЦЕЛЬ РАБОТЫ

1. Ознакомиться с программированием периферийных устройств на примере ввода данных с Web-камеры с использованием библиотеки OpenCV.

2. ВАРИАНТ ЗАДАНИЯ

Реализовать программу с использованием библиотеки OpenCV, которая выполняет следующие действия:

1. Получает поток видеоданных с камеры и выводит его на экран.
2. Выполняет произвольное преобразование изображения (перевод в оттенки серого методом прямого доступа к пикселям с предварительным изменением разрешения до 500×500).
3. Измеряет количество кадров, обрабатываемое программой в секунду (FPS).
4. Оценивает долю времени, затрачиваемого процессором на обработку (ввод, преобразование, показ) видеоданных, получаемых с камеры.

Параметры исследования:

- Искусственная задержка между кадрами: `waitKey(1)` (минимальная задержка для оценки максимальной нагрузки на CPU).
- Метод преобразования: ручное вычисление среднего арифметического каналов BGR во вложенных циклах.

4. ЛИСТИНГ ПРОГРАММЫ

```cpp
#include <opencv2/opencv.hpp>
#include <iostream>

using namespace cv;
using namespace std;

int main() {

    VideoCapture capture(0);

    if (!capture.isOpened()) {
        cerr << "Ошибка: не удалось открыть камеру!" << endl;
        return -1;
    }

    Mat frame;

    // Переменные для расчета FPS
    int frameCount = 0;
    double fps = 0.0;
    double t0 = (double)cv::getTickCount();

    double frameProcessStart = 0;
    double frameProcessEnd = 0;
    double processTimeSec = 0;

    double time1, time2, time3;
    double timeInput, timeProcess, timeOutput;


    while (true) {
        // Засекаем время начала обработки
        frameProcessStart = (double)cv::getTickCount();

        capture >> frame;

        time1 = (double)cv::getTickCount();

        if (frame.empty()) {
            cerr << "Пустой кадр. Завершение." << endl;
            break;
```

```
    // Уменьшаем разрешение для снижения нагрузки
    cv::resize(frame, frame, cv::Size(500, 500));


    // 2. Произвольное преобразование: перевод в оттенки серого вручную
    for (int y = 0; y < frame.rows; y++) {
        for (int x = 0; x < frame.cols; x++) {

            uchar b = frame.at<cv::Vec3b>(y, x)[0];
            uchar g = frame.at<cv::Vec3b>(y, x)[1];
            uchar r = frame.at<cv::Vec3b>(y, x)[2];

            uchar gray_value = (b + g + r) / 3;

            // Записываем среднее значение во все три канала
            frame.at<cv::Vec3b>(y, x)[0] = gray_value;
            frame.at<cv::Vec3b>(y, x)[1] = gray_value;
            frame.at<cv::Vec3b>(y, x)[2] = gray_value;
        }
    }
    time2 = (double)cv::getTickCount();


    // Показываем кадр
    imshow("test", frame);


    // конца обработки
    frameProcessEnd = (double)cv::getTickCount();

    // время обработки одного кадра в сек
    processTimeSec = (frameProcessEnd - frameProcessStart) / cv::getTickFrequency();

    int waitTimeMs = 1;
    char c = (char)waitKey(waitTimeMs);
```

```
        char c = (char)waitKey(waitTimeMs);
        if (c == 27) break; // Выход по ESC

        frameCount++;

        double t1 = (double)cv::getTickCount();
        double elapsedSec = (t1 - t0) / cv::getTickFrequency();


        // в секундах
        timeInput= (time1 - frameProcessStart) / cv::getTickFrequency();
        timeProcess = (time2 - time1) / cv::getTickFrequency();
        timeOutput = (frameProcessEnd - time2) / cv::getTickFrequency();

        if (elapsedSec >= 1.0) {
            fps = frameCount / elapsedSec;
            frameCount = 0;
            t0 = t1;

            double processTimeMs = processTimeSec * 1000.0;
            double totalFrameTimeMs = processTimeMs + waitTimeMs;
            double LoadPercent = (processTimeMs / totalFrameTimeMs) * 100.0;


           // Переводим все времена в миллисекунды
            double timeInput_ms = timeInput * 1000.0;
            double timeProcess_ms = timeProcess * 1000.0;
            double timeOutput_ms = timeOutput * 1000.0;

            double percent_input = (timeInput_ms / totalFrameTimeMs) * 100.0;
            double percent_process = (timeProcess_ms / totalFrameTimeMs) * 100.0;
            double percent_output = (timeOutput_ms / totalFrameTimeMs) * 100.0;

            cout << "FPS: " << fps << '\n';
            cout << "Processing time: " << processTimeSec << " sec (" << processTimeMs << " ms)\n";
            cout << "The share of time spent on processing: " << LoadPercent << "%\n";
            cout << "time Input" << percent_input << "%\n";
            cout << "time process" << percent_process << "%\n";
            cout << "time output" << percent_output << "%\n";
        }
    }
```

```
    capture.release();
    destroyAllWindows();

    return 0;
}
```

5. ОПИСАНИЕ РАБОТЫ

В ходе выполнения лабораторной работы была разработана программа для захвата и обработки видеопотока в реальном времени.

Основные этапы работы программы:

1. **Инициализация:** С помощью класса `VideoCapture` открывается видеопоток с камеры по умолчанию.

2. **Захват кадра (Ввод):** На каждой итерации цикла считывается кадр. Замеряется время, затрачиваемое на эту операцию.
3. **Преобразование:**
    a. Разрешение кадра уменьшается до 500×500 пикселей функцией `cv::resize` для оптимизации.
    b. Реализован **прямой доступ к пикселям** через метод `at<cv::Vec3b>(y, x)`. Для каждого из 250 000 пикселей считываются значения каналов B, G, R, вычисляется их среднее арифметическое, и результат записывается обратно во все три канала.
4. **Показ:** Обработанный кадр выводится в окно с помощью `imshow`.
5. **Измерение производительности:** Используется высокоточный таймер OpenCV `cv::getTickCount()`. Время цикла разделяется на три компонента (ввод, преобразование, показ), на основе которых рассчитывается процентное соотношение нагрузки на процессор.

Результаты измерений *(при задержке `waitKey(1)`)*:

Параметр	Среднее значение
FPS (кадров в секунду)	~45 - 55
Общее время обработки	~15 - 20 мс
Общая загрузка CPU	~94 - 96%
Доля времени: Ввод	~10 - 15%
Доля времени: Преобразование	~70 - 80%
Доля времени: Показ	~5 - 10%

Анализ результатов: Наибольшую долю процессорного времени (70-80%) занимает этап **преобразования**. Это обусловлено тем, что ручной перебор пикселей во вложенных циклах `for` с многократными вызовами метода `.at<>()` является вычислительно сложной операцией по сравнению с оптимизированными внутренними функциями OpenCV (такими как `cv::cvtColor`). Использование минимальной задержки `waitKey(1)` сводит время простоя процессора к минимуму, в результате чего общая загрузка CPU приближается к 95-100%, а FPS ограничивается исключительно вычислительной мощностью процессора, а не искусственной задержкой.

6. КОМАНДЫ ДЛЯ КОМПИЛЯЦИИ

```
g++ -g C:\Users\Yarcka\labs_ecm\lab5\lab5.cpp -o
C:\Users\Yarcka\labs_ecm\lab5\lab5.exe -I
C:/Users/Yarcka/opencv/install/include -L
C:/Users/Yarcka/opencv/install/x64/mingw/lib -lopencv_core500 -
lopencv_highgui500 -lopencv_imgproc500 -lopencv_videoio500 -
lopencv_imgcodecs500

.\lab5.exe
```

7. ВЫВОД ПО РЕЗУЛЬТАТАМ ЛАБОРАТОРНОЙ РАБОТЫ

В ходе выполнения лабораторной работы были достигнуты следующие результаты:

1. Освоены принципы высокоуровневой работы с периферийными устройствами на примере веб-камеры с использованием библиотеки OpenCV. Реализован стабильный захват видеопотока и его отображение в окне.
2. Выполнено произвольное преобразование изображения (перевод в оттенки серого) методом прямого доступа к матрице пикселей `cv::Mat`. Это позволило на практике изучить структуру хранения цветного изображения (3 канала BGR) и механизм поэлементного доступа к данным через метод `.at<cv::Vec3b>()`.
3. Произведена детальная оценка производительности программы: измерено количество кадров в секунду (FPS) и рассчитана доля процессорного времени, затрачиваемого на каждый из трех этапов (ввод, преобразование, показ).
4. Экспериментально подтверждено, что поэлементная обработка пикселей в циклах является наиболее ресурсоемкой операцией. При минимальной задержке `waitKey(1)` программа загружает процессор почти на 100%, что наглядно демонстрирует прямую зависимость нагрузки CPU от частоты опроса видеопотока и сложности алгоритма обработки.